

EXAMINATION ASSIGNMENT

Website Archive Submitter & Automated Backup Repository

Domain Discovery | URL Collection | Archive Submission | Repository | Automation

1. Objective

Build a Python-based website archiving tool that accepts one or more domains, discovers the URLs belonging to those domains, and automatically submits the discovered public URLs to supported web-archiving services. The system should maintain a permanent internal record of what was discovered, what was submitted, when it was submitted, and the resulting archive information.

2. Core Concept

The tool should work as an automated website backup and archival pipeline.

Basic workflow:

1. User adds one domain.
2. System discovers the domain's accessible links.
3. System filters and normalizes the URLs.
4. System creates a submission queue.
5. URLs are submitted one by one to supported archival services.
6. Submission status is recorded.
7. Archive URLs or identifiers are stored in the repository.
8. The system continues until the discovered queue has been processed.

3. Single Domain Support

The minimum version must support a single-domain project.

- Enter domain/website URL.
- Start discovery.
- Display discovered URLs.
- Show total URLs found.
- Start archival submission.
- Show progress.
- Store successful and failed submissions.
- Allow the process to resume after interruption.

4. Multi-Domain Support

The architecture should also support multiple domains. A user should be able to add several websites and process them independently or through a common queue.

- Multiple domain projects

- Domain-level status
- Independent URL queues
- Combined processing queue
- Per-domain statistics
- Per-domain archive history

5. URL Discovery

When a domain is added, the system must attempt to discover as many relevant public URLs as practical.

Investigate and combine appropriate discovery methods such as:

- Internal HTML links
- Sitemap.xml
- Sitemap indexes
- Robots.txt references
- Canonical URLs
- Pagination
- Relevant feed URLs
- Other publicly accessible site navigation structures

The crawler should remain within the configured domain unless the user explicitly enables an appropriate external-link handling mode.

6. URL Processing

Before submission, the system should process discovered URLs.

- Normalize URLs.
- Remove obvious duplicate URLs.
- Handle URL fragments appropriately.
- Identify redirects where practical.
- Record HTTP status where available.
- Identify inaccessible or failed URLs.
- Maintain the original discovered URL.

7. Archive Services

The core assignment should support submission to public web-archiving services where their published interfaces and terms permit automated submission.

Primary targets:

- Internet Archive / Wayback Machine
- Archive.today / Archive.is

Students must investigate the currently available submission mechanism for each service and implement the integration without bypassing CAPTCHAs, authentication controls, rate limits or other security mechanisms.

8. Automated Submission Queue

The system should not require the user to manually submit every URL.

- Create a persistent queue.
- Submit URLs sequentially or using controlled concurrency where permitted.
- Record submission start and completion times.
- Record success/failure.
- Store returned archive URLs or identifiers when available.
- Retry temporary failures using controlled retry logic.
- Continue processing after individual failures.

9. Repository

The application should maintain its own repository containing the discovered website structure and archival history.

At minimum, store:

- Domain
- Original URL
- Normalized URL
- Discovery source
- Discovery timestamp
- Submission service
- Submission timestamp
- Submission status
- Archive URL
- Archive identifier, where available
- HTTP status
- Error message, where applicable
- Last attempted time

10. Incremental Backup

The tool should not submit the same URL repeatedly without a reason.

- Remember previously discovered URLs.
- Identify newly discovered URLs.
- Identify URLs that have never been archived by the tool.
- Allow re-archiving when the user explicitly requests an updated snapshot.
- Maintain a history of multiple archive submissions for the same URL.

11. Website Changes

A strong implementation should allow the same domain to be scanned again later. The system should identify new pages and changes in the website's URL inventory.

Example: 5,000 URLs were discovered during the first scan. A month later, 350 new URLs are discovered. The system should be able to process those new URLs without rebuilding the entire repository.

12. Dashboard

Create a dashboard showing:

- Domains
- Total URLs discovered
- URLs queued
- URLs submitted
- Successful submissions
- Failed submissions
- URLs pending
- Archive service used
- Archive links
- Last scan time
- Last submission time
- Current processing status

13. Search & Repository

The user should be able to search the internal repository by domain or URL and determine whether the URL has already been submitted.

- Search domain
- Search URL
- Filter by archive service
- Filter by submission status
- Open stored archive URL
- View submission history

14. Large Website Handling

The system should be designed for websites containing thousands or significantly more URLs.

- Use a database instead of keeping the complete crawl in memory.
- Use queues or background workers.
- Process URLs in batches.
- Prevent duplicate crawling.
- Avoid repeatedly downloading the same page unnecessarily.
- Allow the process to resume after a crash or server restart.

15. JavaScript-Rendered Websites

Some websites do not expose all useful content through the initial HTML response. Students should investigate how JavaScript-rendered pages affect URL discovery and archival quality.

16. Instagram Website Archiving Bonus

As a bonus challenge, students may investigate websites or publicly accessible pages whose content requires JavaScript rendering, including Instagram pages where technically and legally appropriate.

The goal is to determine whether a browser-rendering solution can load the page, capture the resulting rendered content, and submit or preserve the resulting page through an appropriate archival workflow.

Students may investigate technologies such as Playwright or Selenium for legitimate browser rendering. They must not bypass login requirements, CAPTCHAs, access restrictions or other security controls.

17. Performance & Automation

Students should investigate how the system can process a large URL inventory efficiently.

- Measure crawl speed.
- Measure submission throughput.
- Identify slow services or URLs.
- Use background processing.
- Use controlled concurrency where permitted.
- Implement retry and backoff.
- Display real-time queue progress.

18. Failure Recovery

- A server restart must not lose the queue.
- A failed URL must not stop the complete project.
- The system should record the reason for failure.
- Interrupted jobs should resume from the last known state.
- Repeated failures should remain visible for manual review.

19. Mandatory Demonstration

- Add one domain.
- Automatically discover its URLs.
- Show the discovered URL inventory.
- Create the submission queue.
- Submit URLs to the supported archival service(s).
- Show successful and failed submissions.
- Show stored archive URLs.
- Interrupt the process and demonstrate resume capability.
- Run a second scan and demonstrate duplicate/new URL handling.
- Demonstrate multi-domain architecture or processing.

20. Required Deliverables

- Complete source code
- Working web interface
- Crawler/discovery engine
- Archive submission integrations
- Database/schema
- Submission queue
- Dashboard

- Technical architecture documentation
- Setup instructions
- Demonstration video

21. Bonus Challenges

- Support more than two archive services.
- Support 50+ domains simultaneously.
- Handle very large websites with hundreds of thousands of URLs.
- JavaScript-rendered page capture using a browser automation engine.
- Instagram or other JS-heavy public-page archival workflow.
- Scheduled recurring scans.
- Automatic detection of newly added URLs.
- Change detection before re-archiving.
- Archive comparison between two snapshots.
- Distributed workers.
- Priority queues for important URLs.
- Automatic sitemap discovery and refresh.
- Export complete archival inventory to CSV/JSON.

22. Expected Outcome

The finished system should operate as a website archival repository. A user should be able to add one domain or multiple domains, allow the system to discover the site's public URLs, automatically submit those URLs to supported archive services, and maintain a searchable historical record of the entire process.

Examination Focus: Crawling • Automation • Archival APIs • Queues • Databases • Browser Rendering